NODUS

Feature Testing Playbook

Enterprise Dependency Intelligence — NEXUS Demo Environment

Purpose: a tester-executable playbook covering 12 feature areas of the NODUS application (graphhacks repository, NEXUS demo dataset). Each test case lists exact tester actions, expected behavior, negative/security checks, and the evidence required to call the feature a PASS. All endpoints, demo tokens, source IDs, and entity IDs in this document were verified directly against the backend source code (backend/app/main.py, backend/app/governance/, backend/tests/) prior to publication; no endpoint or credential is invented.

Repository: https://github.com/Glassman007/NODUS • Local path: G:\graphhacks

Document generated for execution against a locally running stack: Neo4j, PostgreSQL, FastAPI backend (port 8001), Vite frontend (port 5174), NEXUS demo mode enabled.

Contents

1. Purpose and Scope
2. Environment Setup (Windows PowerShell)
3. Demo User Credentials
4. Test Cases TC-01 through TC-12
5. Execution Checklist (PASS / FAIL / BLOCKED / NOT RUN)

1. Purpose and Scope

NODUS is an investigation and decision-support system that connects enterprise knowledge (service catalogs, interface registries, architecture decisions, repositories, issue trackers) into a permission-aware Neo4j knowledge graph, and uses a Gemini-backed multi-agent coordinator to run objective-driven investigations, verify claims against evidence, and surface dependency risk. This playbook exercises the system end-to-end against the fictional NEXUS Technologies demo organization using the project's built-in demo mode (`GRAPH_HACKS_NEXUS_DEMO_MODE=true`).

Each test case below is written against functionality actually present in the repository at the time of writing (commit confirmed via `git log -1` at execution time, recorded in the execution checklist). Where the original feature brief described an endpoint, token, or ID that did not match the code, it has been corrected here and the discrepancy is noted inline.

2. Environment Setup (Windows PowerShell)

Verified against `docker-compose.yml`, `frontend/package.json`, `frontend/vite.config.ts`, and the existing `.env` in the repo root.

Step 1 — Start Neo4j and PostgreSQL via Docker Compose

docker-compose.yml defines two services: `postgres` (postgres:16.10-alpine, port 5432) and `neo4j` (neo4j:5.26.32-community, ports 7474/7687). Both require `POSTGRES_PASSWORD` and `NEO4J_PASSWORD` to be set in `.env` (already present in this repo's local `.env`; never commit it).

```
cd G:\graphhacks
docker compose up -d
docker ps # confirm graphhacks-postgres-1 and graphhacks-neo4j are healthy
```

Step 2 — Start the FastAPI backend on port 8001 with NEXUS demo mode

The backend entrypoint is `backend.app.main:app`, served via uvicorn (confirmed in scripts/run_neo4j_e2e.py and scripts/run_connector_pipeline_e2e.py, which both invoke `python -m uvicorn backend.app.main:app`). Demo mode is gated by the `GRAPH_HACKS_NEXUS_DEMO_MODE` environment variable, consumed by `backend/app/governance/identity.py` (`isolated_demo_mode_enabled()`).

```
cd G:\graphhacks
.venv\Scripts\Activate.ps1
$env:GRAPH_HACKS_NEXUS_DEMO_MODE = "true"
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8001 --reload
```

Step 3 — Start the Vite frontend

frontend/package.json defines `"dev": "vite"`. Vite's default dev port is 5173; if that port is occupied (as it was during this execution), Vite automatically falls back to the next free port (5174), which is also the port referenced in the feature brief. `vite.config.ts` proxies all `/api` requests to `http://127.0.0.1:8001`, so the backend must already be running.

```
cd G:\graphhacks\frontend
npm install
npm run dev
# Vite prints the actual bound port (5173 or next free port, e.g. 5174)
```

Verify the stack: GET http://127.0.0.1:8001/docs should return the FastAPI Swagger UI (HTTP 200); GET http://127.0.0.1:8001/api/v1/nexus/session with header `Authorization: Bearer 4` should return Leena Vardhan's ADMIN session JSON.

3. Demo User Credentials

All tokens below are defined in `backend/app/governance/local_demo_login.py` and only resolve when demo mode is enabled. The feature brief's original tokens were confirmed correct against this file; aliases (e.g. `dev-02-pay`) are translated to canonical tokens (e.g. `D02`) by `backend/app/governance/demo_tokens.py` before lookup.

Token	Canonical ID	Employee	Role	Project(s)
4	4	Leena Vardhan (NEX-EMP-001)	ADMIN	All (PAY, COM, ID, DATA)
dev-02-pay	D02	Arav Mehta (NEX-EMP-002)	DEVELOPER	NEX-PRJ-PAY
intern-10-com	I10	Anika Bose (NEX-EMP-010)	INTERN	NEX-PRJ-COM

All three tokens above were confirmed live: each returns HTTP 200 with the expected employee_id, display_name, and role_code from GET /api/v1/nexus/session. An invalid/unknown token and a missing Authorization header both return HTTP 401, confirmed live.

No real Gemini API key, database password, or non-demo secret is reproduced anywhere in this document.

4. Test Cases

Unless stated otherwise, all requests use base URL `http://127.0.0.1:8001` and header `Authorization: Bearer <token>`. UI steps assume the frontend is reachable at `http://127.0.0.1:5174/` (or whatever port Vite printed).

TC-01 — Role-Based Authentication

Tester actions

- GET /api/v1/nexus/session with header Authorization: Bearer 4 (Admin).
- Repeat with Bearer dev-02-pay (Developer) and Bearer intern-10-com (Intern).
- Repeat with Authorization: Bearer not-a-real-token.
- Repeat with no Authorization header at all.

Expected successful behavior

- Admin token returns role_code=ADMIN, display_name=Leena Vardhan, project_ids containing all four NEX-PRJ-* projects.
- Developer token returns role_code=DEVELOPER, display_name=Arav Mehta, project_ids=[NEX-PRJ-PAY].
- Intern token returns role_code=INTERN, display_name=Anika Bose, project_ids=[NEX-PRJ-COM].

Negative / security checks

- Invalid token must return HTTP 401 Invalid credentials, never a 200 or a 500.
- Missing Authorization header must return HTTP 401 Bearer token required.

Evidence required for PASS

Raw HTTP response bodies and status codes for all five calls above (3 valid + 1 invalid + 1 missing).

TC-02 — Project-Specific Authorization

Tester actions

- GET /api/v1/nexus/sources and GET /api/v1/nexus/authorization-scope as Intern (intern-10-com).
- Repeat both calls as Admin (token 4) and compare source_ids lists.
- As Intern, GET /api/v1/workspaces/nexus-workspace/entities/NEX-SVC-PAYMENT_API and inspect the provenance[] array for restricted source_ids (e.g. NEX-SRC-PAY-ISSUES, NEX-SRC-ACCESS_GOVERNANCE).
- Repeat the same entity fetch as Admin and compare provenance length/content.

Expected successful behavior

- Intern's /sources and /authorization-scope responses list only COMPANY_SHARED/FOUNDATIONAL sources (NEX-SRC-ARCHITECTURE_DECISIONS, NEX-SRC-COMPANY_FOUNDATION, NEX-SRC-INTERFACE_REGISTRY, NEX-SRC-KNOWLEDGE_HUB, NEX-SRC-SERVICE_CATALOG) plus nothing project- or admin-restricted.
- Admin's lists include all 28 sources across all four projects plus admin-only governance sources.
- Entity-level enforcement is server-side: the same entity_id returns different, permission-filtered provenance content per caller, not merely a UI toggle.

Negative / security checks

- Intern must never see NEX-SRC-PAY-*, NEX-SRC-ID-*, NEX-SRC-DATA-*, or NEX-SRC-ACCESS_GOVERNANCE / NEX-SRC-PRIVILEGED_CHANGES content in any response field.

Evidence required for PASS

Side-by-side JSON diffs of the Intern vs Admin /sources, /authorization-scope, and entity provenance[] responses.

TC-03 — Source Access Requests (full lifecycle)

Tester actions

- As Intern (intern-10-com), POST /api/v1/nexus/access-requests with body {resource_id, requested_actions, justification}. Note: the field is resource_id (NOT source_id) and requested_actions must be one of ADMIN/DELETE/EXECUTE/EXPORT/READ/WRITE.
- GET /api/v1/nexus/access-requests/mine as the same Intern and confirm the new request appears PENDING.
- As Developer (dev-02-pay), GET /api/v1/nexus/access-requests/pending and POST .../{request_id}/approve — both must be rejected.
- As Admin (token 4), GET /api/v1/nexus/access-requests/pending, then POST .../{request_id}/approve.
- GET /api/v1/nexus/grants/active as Admin and confirm a new ACTIVE grant for the requesting employee/resource.
- As Admin, POST /api/v1/nexus/grants/{grant_id}/revoke to clean up, then re-check /grants/active shows status REVOKED and it is no longer returned as active.

Expected successful behavior

- Request creation returns status=PENDING with a generated request_id (format AR-<uuid hex>).
- Approval as Admin returns a composite {request, decision, grant} object; the grant's status is ACTIVE.
- Revocation returns the grant with status=REVOKED and a non-null revoked_at.

Negative / security checks

- /access-requests/pending and the approve/deny/revoke endpoints must return HTTP 403 for any non-ADMIN token (Developer and Intern).

Evidence required for PASS

Request/approve/revoke JSON bodies, the 403 responses for non-admin attempts, and confirmation the grant was revoked (no permanent state left behind).

TC-04 — Objective-Driven Investigations ("Try an Investigation" UI flow)

Tester actions

- Log into the frontend as Developer (dev-02-pay), switch to the NEX-PRJ-PAY project.
- Select the pre-built scenario "Duplicate Payment Prevention" (frontend/src/workspace/scenarios.ts, id developer-duplicate-payment-prevention).
- Confirm its required source IDs are pre-selected: NEX-SRC-SERVICE_CATALOG, NEX-SRC-INTERFACE_REGISTRY, NEX-SRC-ARCHITECTURE_DECISIONS, NEX-SRC-PAY-REPO, NEX-SRC-PAY-ISSUES.
- Submit the investigation and observe the run through to completion or a graceful INCOMPLETE/BLOCKED state.

- Also try the Admin scenario "Authentication API Change Impact" and the Intern scenario "Understanding Checkout Dependencies" (NEX-PRJ-COM) for role coverage.

Expected successful behavior

- The UI submits POST /api/v1/workspaces/nexus-workspace/investigations with objective + selected_source_ids + project_id exactly matching the scenario definition.
- A run_id is returned and the run appears in GET .../investigations history.
- On a healthy model provider, status becomes COMPLETE with findings, structured_findings, and verification_status populated.

Negative / security checks

- Selecting a source the role is not authorized for must not silently widen the investigation scope beyond the server-computed intersection of selected ∩ authorized sources.

Evidence required for PASS

A captured run_id plus its investigation detail response showing COMPLETE status and non-empty findings/evidence_registry.

Correction note

Source IDs NEX-SRC-SERVICE_CATALOG, NEX-SRC-INTERFACE_REGISTRY, and NEX-SRC-ARCHITECTURE_DECISIONS exist and are correct. NEX-SRC-PAY-REPO and NEX-SRC-PAY-ISSUES also exist and are correct for this scenario, confirmed in scenarios.ts.

TC-05 — Neo4j Organizational Graph

Tester actions

- GET /api/v1/workspaces/nexus-workspace/entities/NEX-SVC-PAYMENT_API
- GET .../entities/NEX-SVC-PAYMENT_API/neighborhood?limit=50
- GET .../entities/NEX-SVC-PAYMENT_API/evidence?limit=20
- GET .../entities/NEX-SVC-PAYMENT_API/history and .../assertions

Expected successful behavior

- Entity response includes id, type=SERVICE, name=payment-api, and a provenance[] array citing real record IDs (e.g. NEX-ADR-001, NEX-DOC-009).
- Neighborhood returns connected nodes/relationships (e.g. OWNS, DEPENDS_ON) without error.
- Evidence endpoint returns source_id/record_id/title/observed_at tuples that match the entity's provenance.

Negative / security checks

An unauthorized caller's evidence/provenance content must be filtered exactly as in TC-02, not merely the entity's existence.

Evidence required for PASS

Full JSON bodies for all four calls against NEX-SVC-PAYMENT_API, as Admin.

TC-06 — Multi-Hop Dependency Tracing

Tester actions

- GET /api/v1/workspaces/nexus-workspace/paths?source_id=NEX-SVC-PAYMENT_API■_id=NEX-SVC-ORDER_API&max;_depth=4&limit;=5 as Admin.

- Repeat with a shallower max_depth (e.g. 1) and confirm fewer or no paths are returned, demonstrating depth actually bounds the traversal.

Expected successful behavior
Response includes a paths[] array; each path has an ordered nodes[] and relationships[] list connecting the two services through the graph.

Negative / security checks
Path traversal must not surface node/relationship content from sources the caller is not authorized to see.

Evidence required for PASS
The returned paths JSON, with at least one non-trivial (length > 1) path, plus the depth-limited comparison.

TC-07 — Gemini-Powered Investigation

Tester actions
- Submit a real investigation (TC-04 flow) and capture the run's model_usage block: llm_calls, total_input_tokens, total_output_tokens, model_used, estimated_cost_usd.
- Confirm GEMINI_API_KEY is non-empty in the running backend's environment before testing (never print the key itself).

Expected successful behavior
model_usage.llm_calls >= 1, model_used is a real configured Gemini model id (GEMINI_MODEL_LIGHTWEIGHT or GEMINI_MODEL_DEFAULT from .env), and total_output_tokens > 0 for a COMPLETE run.

Negative / security checks
If the Gemini provider returns a quota/429 or 503/UNAVAILABLE error, the investigation must fail closed to an INCOMPLETE/BLOCKED status with an honest error_message, never a fabricated COMPLETE result.

Evidence required for PASS
A model_usage block with non-zero output tokens and a real run_id, OR, if the provider is unavailable, the raw error_message proving graceful degradation (this would be scored BLOCKED, not PASS, since the LLM path itself was not exercised end-to-end).

TC-08 — Multi-Agent Investigation Coordination

Tester actions
- Inspect the detail response of a COMPLETE run for skill_used, tool_calls[] (each with tool_id/arguments/result), and any specialist/verifier-role metadata.
- Confirm backend/app/agent/coordinator.py's per-pass RuntimeLimits (max_llm_calls, max_tool_calls, overall_timeout_seconds) were respected (tool_calls count <= limit).

Expected successful behavior
skill_used is populated from the skill registry, tool_calls[] shows at least one real tool invocation (e.g. a graph neighborhood or evidence lookup), and a verifier pass produces verification_status/verification_summary distinguishing VERIFIED from UNVERIFIED claims.

Negative / security checks
The coordinator must stop within its configured budget (<=10 LLM calls, <=8 tool calls, <=150s per pass by default) rather than looping indefinitely.

Evidence required for PASS

The full tool_calls[] and skill_used fields from a COMPLETE run. A run that never leaves llm_calls=0/tool_calls=0 due to provider unavailability does not satisfy this test case.

TC-09 — Investigation Reports

Tester actions

- From a COMPLETE run, inspect summary, full_analysis, structured_findings[], evidence_refs[], and verification_status.
- Confirm each structured_findings[] entry has a claim, claim_type, verification_status, and evidence_ids/entity_ids where applicable.
- Confirm any claim referencing an entity/source the requester did not select or is not authorized for is flagged, not silently included as verified.

Expected successful behavior

A human-readable summary plus a structured, evidence-linked findings list, with verification_status reflecting genuine claim-to-evidence checking rather than always defaulting to UNVERIFIED.

Negative / security checks

A report must never claim verification_status=VERIFIED for a claim with an empty evidence_ids[] list.

Evidence required for PASS

A full investigation report JSON from a COMPLETE run with at least one VERIFIED finding backed by a real evidence_id.

TC-10 — Interactive Graph Visualization

Tester actions

- Open the frontend, log in with any token, and navigate to the Knowledge & Dependencies / graph view.
- Select a node (e.g. the payment-api service), pan and zoom the canvas, and open its neighborhood.
- Confirm the labels and relationship types shown match the backend's /neighborhood response for the same entity.

Expected successful behavior

The graph renders using Cytoscape (frontend dependency cytoscape ^3.34.4), supports zoom/pan/node selection, and node/edge labels match live backend data exactly.

Negative / security checks

The graph must not render nodes/edges sourced from content the logged-in role is not authorized to see.

Evidence required for PASS

Screenshots or a recorded interaction showing zoom, pan, node selection, and a side-by-side label match against the API response.

TC-11 — FastAPI Investigation Endpoints

Tester actions

- Open http://127.0.0.1:8001/docs and confirm Swagger UI loads and lists the NEXUS and workspace routes.
- GET /api/v1/session and GET /api/v1/nexus/session with a valid token.

- POST /api/v1/workspaces/nexus-workspace/investigations, then GET the same path (list) and GET .../investigations/{run_id} (detail).
- Repeat the POST/GET calls with no Authorization header and confirm rejection.

Expected successful behavior

Swagger UI returns HTTP 200; /api/v1/session returns the principal's subject/tenant_id/permissions; investigation POST returns a run_id that is retrievable from both the list and detail endpoints.

Negative / security checks

All of the above must return HTTP 401 without a valid bearer token.

Evidence required for PASS

HTTP status codes and bodies for every call above, including the unauthorized-access 401s.

TC-12 — NEXUS Dataset

Tester actions

- Confirm Neo4j and PostgreSQL containers are healthy (docker ps) and reachable.
- Run a direct Cypher count (e.g. MATCH (n) RETURN count(n)) against Neo4j to confirm the graph is populated.
- Confirm the four NEXUS projects exist: NEX-PRJ-PAY (Payment), NEX-PRJ-COM (Commerce), NEX-PRJ-ID (Identity), NEX-PRJ-DATA (Data), each with REPO/ISSUES/CI/DISCUSS sources (backend/app/governance/local_demo_login.py).
- Confirm a cross-project dependency exists by tracing a path from a PAY entity to a COM/ID entity (TC-06).
- Confirm every entity in TC-05's response carries source provenance (source_id + record_id), never an unsourced claim.

Expected successful behavior

Neo4j reports a non-zero node count; PostgreSQL has the expected application tables (e.g. nexus_sources, nexus_roles, nodus_investigation_runs); all four projects and their sources are present and queryable.

Negative / security checks

No dataset health check should require disabling authorization or bypassing demo mode.

Evidence required for PASS

Neo4j node count, PostgreSQL table listing, and the four-project source listing from an Admin /sources call.

5. Execution Checklist

To be completed during an actual execution pass. Evidence tiers, from weakest to strongest: code exists < unit test passes < integration test passes < UI test passes < full live end-to-end pass. Only assign PASS when evidence at the tier required by the test case's acceptance criteria was personally observed.

ID	Feature	Result (PASS/FAIL/BLOCKED/NOT RUN)	Evidence Ref	Notes
TC-01	Role-Based Authentication			
TC-02	Project-Specific Authorization			
TC-03	Source Access Requests			
TC-04	Objective-Driven Investigations			
TC-05	Neo4j Organizational Graph			
TC-06	Multi-Hop Dependency Tracing			
TC-07	Gemini-Powered Investigation			
TC-08	Multi-Agent Investigation Coordination			
TC-09	Investigation Reports			
TC-10	Interactive Graph Visualization			
TC-11	FastAPI Investigation Endpoints			
TC-12	NEXUS Dataset			

Tester signature: _____ Date: _____ Repo commit (git log -1): _____

Repository HEAD at time of writing: efe32958834de31176815cce1ccfc7b955162636 2026-11-04 00:30:52 +0530